

Eksperimen 7 — Optimasi Side View

Notebook ini memuat dan memvisualisasikan hasil training & evaluasi Eksperimen 7.

Target utama eksperimen ini adalah **meningkatkan Recall pada kelas minoritas (safe_driving) khusus untuk model Side View**, serta mencegah overfitting akibat redundansi temporal frame.

Model	EfficientNetV2-S (Binary Classification)
View	Front + Side
Fusion	Average & Adaptive (confidence-based)
Versi Eksperimen	Eksperimen 7 (Optimasi Side View, Asymmetric Freeze, static Class Weights, Label Smoothing)

In [1]:

```
# — Setup —  
import json  
import sys  
from pathlib import Path  
  
import matplotlib.pyplot as plt  
import matplotlib.gridspec as gridspec  
import numpy as np  
import pandas as pd  
import seaborn as sns  
  
# Tambahkan root project ke path agar bisa import src.*  
PROJECT_ROOT = Path("../").resolve()  
if str(PROJECT_ROOT) not in sys.path:  
    sys.path.insert(0, str(PROJECT_ROOT))
```

```
RESULTS_DIR = PROJECT_ROOT / "results"
```

```
# Style
```

```
plt.style.use("seaborn-v0_8-darkgrid")
```

```
COLORS = {"front": "#4C9BE8", "side": "#E8834C", "avg": "#6FCB9F", "top": "#F08080", "bottom": "#4682B4"}
```

```
print("✓ Setup selesai. Project root:", PROJECT_ROOT)
```

✓ Setup selesai. Project root: D:\Skripsi\Experiment

0. Perbandingan Konfigurasi (Eksperimen 5 → Eksperimen 6 → Eksperimen 7)

Tabel perbandingan parameter untuk melihat arah optimasi:

In [2]:

```
# — Tampilkan diff konfigurasi 3 eksperimen terakhir —————
```

```
config_diff = [
    {"Parameter": "WEIGHT_DECAY", "Exp 5 (Baseline)": "5e-4", "Exp 6 (Baseline)": "1e-3", "Exp 7 (Optimasi)": "1e-2"},
    {"Parameter": "NUM_STAGES_TO_FREEZE", "Exp 5 (Baseline)": "4", "Exp 6 (Baseline)": "8", "Exp 7 (Optimasi)": "16"},
    {"Parameter": "LABEL_SMOOTHING", "Exp 5 (Baseline)": "0.0", "Exp 6 (Baseline)": "0.1", "Exp 7 (Optimasi)": "0.2"},
    {"Parameter": "CLASS_WEIGHTS", "Exp 5 (Baseline)": "Dinamis", "Exp 6 (Baseline)": "Dinamis", "Exp 7 (Optimasi)": "Dinamis"},
    {"Parameter": "FRAME_STRIDE", "Exp 5 (Baseline)": "5", "Exp 6 (Baseline)": "5", "Exp 7 (Optimasi)": "5"}
]
```

```
df_config = pd.DataFrame(config_diff)
display(df_config.style
        .set_properties(**{"text-align": "left"})
        .set_caption("Perbandingan Konfigurasi Parameter Eksperimen 5, 6, dan 7")
        .hide(axis="index"))
```

Parameter	Exp 5 (Baseline)	Exp 6 (Ablasi)	Exp 7 (Optimasi)	Alasan
WEIGHT_DECAY	5e-4	1e-3	1e-2 (Lebih Kuat)	Menekan magnitude logit

NUM_STAGES_TO_FREEZE	4	5	Front: 5, Side: 3	Asimetris: S view butuh belajar visu oklusi lebih awal
LABEL_SMOOTHING	0.0	0.0	0.1	Mengurang overconfide model
CLASS_WEIGHTS	Dinamis	nan	Statis [5.15, 1.0]	Fokus pada peningkatan Recall kelas safe_driving (minoritas)
FRAME_STRIDE	5	5	5	Mengurang redundansi temporal vi

Perbandingan Konfigurasi Parameter Eksperimen 5, 6, dan 7

0.5. Otomatisasi Training & Evaluasi (Google Colab Experience)

Ubah `RUN_RETRAINING` di bawah ini menjadi `True` jika kamu ingin melatih ulang model `M_front` dan `M_side` dengan konfigurasi Eksperimen 7 secara otomatis dengan log mengalir real-time.

In [3]:

```
# ⚠️ PENTING: Set ke True hanya jika ingin melakukan re-training (mem
RUN_RETRAINING = True
```

```
import subprocess
import os
```

```
def run_command_live(cmd_args, cwd=None):
    """Menjalankan perintah OS secara live dan mengalirkan outputnya la
    env = os.environ.copy()
    env["PYTHONIOENCODING"] = "utf-8"
```

```
env["PYTHONUNBUFFERED"] = "1"
```

```
# Menjalankan subprocess
```

```
process = subprocess.Popen(  
    cmd_args,  
    stdout = subprocess.PIPE,  
    stderr = subprocess.STDOUT, # Gabungkan stderr ke stdout  
    text = True,  
    encoding = "utf-8",  
    cwd = cwd,  
    env = env  
)
```

```
# Baca output baris demi baris secara real-time
```

```
while True:  
    output = process.stdout.readline()  
    if output == " " and process.poll() is not None:  
        break  
    if output:  
        print(output, end = " , flush = True)
```

In [4]:

```
if RUN_RETRAINING:
```

```
# Tentukan executable python di dalam virtual environment (.venv)
```

```
python_exe = str(PROJECT_ROOT / ".venv" / "Scripts" / "python.exe")
```

```
if not os.path.exists(python_exe):
```

```
    python_exe = "python" # Fallback jika venv tidak terdeteksi
```

```
print("🦉 Memulai Retraining Model (Eksperimen 7 — Optimasi Side
```

```
print("1/3 [FRONT VIEW] Melatih model (num_stages_to_freeze = 5)
```

```
run_command_live([python_exe, "-m", "src.train", "--view", "front"], cwd
```

```
print("\n2/3 [SIDE VIEW] Melatih model (num_stages_to_freeze = 3)
```

```
run_command_live([python_exe, "-m", "src.train", "--view", "side"], cwd
```

```
print("\n3/3 [FUSION & EVALUATION] Menjalankan fusion...\n")
```

```
run_command_live([python_exe, "-m", "src.fusion"], cwd = PROJECT_F
```

```
print("\n✓ Re-training selesai! Seluruh hasil di folder results/ telah  
else:
```

```
print("❏ Re-training dilewati (RUN_RETRAINING = False).")
print("  Notebook akan memvisualisasikan data yang sudah ada di fo
```

🐦 Memulai Retraining Model (Eksperimen 7 — Optimasi Side View)...

1/3 [FRONT VIEW] Melatih model (num_stages_to_freeze = 5)...

INFO:

=====

INFO: Experiment Configuration

INFO: =====

INFO: View : front

INFO: Optimizer : AdamW

INFO: Learning Rate : 5.0e-06

INFO: Weight Decay : 1.0e-02

INFO: Scheduler : ReduceLROnPlateau

INFO: Dropout : 0.6

INFO: Frozen Stages : 5

INFO: EarlyStopping : patience=5

INFO: Experiment ID : None

INFO: =====

INFO: Device: cuda

INFO: GPU: NVIDIA GeForce RTX 5060 Laptop GPU

INFO: Frame subsampling diterapkan (stride=5): 5976 frame tersisa unt

INFO: DataLoader dibuat: view=front split=train n=5976 batch_size=

INFO: Frame subsampling diterapkan (stride=5): 1249 frame tersisa unt

INFO: DataLoader dibuat: view=front split=val n=1249 batch_size=32

INFO: Frame subsampling diterapkan (stride=5): 1215 frame tersisa unt

INFO: DataLoader dibuat: view=front split=test n=1215 batch_size=3

INFO: Loading pretrained weights from Hugging Face hub (timm/tf_effic

INFO: HTTP Request: HEAD https://huggingface.co/timm/tf_efficientnetv2_s.in21k_ft_in1k

Warning: You are sending unauthenticated requests to the HF Hub. Pleas

WARNING: Warning: You are sending unauthenticated requests to the H

INFO: [timm/tf_efficientnetv2_s.in21k_ft_in1k] Safe alternative available

INFO: Membekukan conv_stem, bn1, dan 5 stage block pertama pada ba

INFO: =====

INFO: STATUS GRADIENT PARAMETER MODEL (FREEZE vs TRAINABL

INFO: =====

INFO: backbone.conv_stem : FROZEN

```

INFO: backbone.bn1      : FROZEN
INFO: backbone.blocks[0] : FROZEN (contains 2 blocks)
INFO: backbone.blocks[1] : FROZEN (contains 4 blocks)
INFO: backbone.blocks[2] : FROZEN (contains 4 blocks)
INFO: backbone.blocks[3] : FROZEN (contains 6 blocks)
INFO: backbone.blocks[4] : FROZEN (contains 9 blocks)
INFO: backbone.blocks[5] : TRAINABLE (contains 15 blocks)
INFO: backbone.conv_head : TRAINABLE
INFO: backbone.bn2      : TRAINABLE
INFO: classifier        : TRAINABLE
INFO: =====
INFO: Model: tf_efficientnetv2_s | Total params: 20,180,050 | Trainable:
INFO: Class weights (CrossEntropyLoss): safe_driving=5.1500, phone_us
INFO: Label smoothing: 0.10
INFO: =====
INFO: Training: view = front | max_epochs = 30 | patience = 5
INFO: =====
INFO: Epoch 01/30 | train_loss=0.8395 | val_loss=0.8648 | val_F1=0.3
INFO: Epoch 02/30 | train_loss=0.7750 | val_loss=0.8325 | val_F1=0.4
INFO: Epoch 03/30 | train_loss=0.7147 | val_loss=0.8064 | val_F1=0.4
INFO: Epoch 04/30 | train_loss=0.6948 | val_loss=0.7970 | val_F1=0.5
INFO: Epoch 05/30 | train_loss=0.6669 | val_loss=0.7683 | val_F1=0.5
INFO: Epoch 06/30 | train_loss=0.6252 | val_loss=0.7472 | val_F1=0.6
INFO: Epoch 07/30 | train_loss=0.6111 | val_loss=0.7340 | val_F1=0.6
INFO: Epoch 08/30 | train_loss=0.5968 | val_loss=0.7159 | val_F1=0.6
INFO: Epoch 09/30 | train_loss=0.5744 | val_loss=0.7132 | val_F1=0.6
INFO: Epoch 10/30 | train_loss=0.5569 | val_loss=0.7064 | val_F1=0.6
INFO: Epoch 11/30 | train_loss=0.5615 | val_loss=0.6845 | val_F1=0.7
INFO: Epoch 12/30 | train_loss=0.5406 | val_loss=0.6938 | val_F1=0.7
INFO: Epoch 13/30 | train_loss=0.5347 | val_loss=0.6864 | val_F1=0.7
INFO: Epoch 14/30 | train_loss=0.5228 | val_loss=0.6823 | val_F1=0.7
INFO: Epoch 15/30 | train_loss=0.5162 | val_loss=0.6681 | val_F1=0.7
INFO: Epoch 16/30 | train_loss=0.5072 | val_loss=0.6617 | val_F1=0.7
INFO: Epoch 17/30 | train_loss=0.4973 | val_loss=0.6629 | val_F1=0.7
INFO: Epoch 18/30 | train_loss=0.4971 | val_loss=0.6646 | val_F1=0.7
INFO: Epoch 19/30 | train_loss=0.4882 | val_loss=0.6623 | val_F1=0.7
INFO: Epoch 20/30 | train_loss=0.4928 | val_loss=0.6513 | val_F1=0.7
INFO: Epoch 21/30 | train_loss=0.4870 | val_loss=0.6525 | val_F1=0.7
INFO: Epoch 22/30 | train_loss=0.4873 | val_loss=0.6587 | val_F1=0.7
INFO: Epoch 23/30 | train_loss=0.4874 | val_loss=0.6489 | val_F1=0.7

```

```
INFO: Epoch 24/30 | train_loss=0.4860 | val_loss=0.6466 | val_F1=0.7
INFO: Epoch 25/30 | train_loss=0.4887 | val_loss=0.6533 | val_F1=0.7
INFO: Epoch 26/30 | train_loss=0.4792 | val_loss=0.6433 | val_F1=0.7
INFO: Epoch 27/30 | train_loss=0.4777 | val_loss=0.6497 | val_F1=0.7
INFO: Epoch 28/30 | train_loss=0.4696 | val_loss=0.6505 | val_F1=0.7
INFO: Epoch 29/30 | train_loss=0.4770 | val_loss=0.6436 | val_F1=0.7
INFO: Epoch 30/30 | train_loss=0.4727 | val_loss=0.6428 | val_F1=0.7
INFO: =====
INFO: Training selesai dalam 29.7 menit. Best epoch: 29 (val_F1=0.793)
INFO: Checkpoint tersimpan di: D:\Skripsi\Experiment\checkpoints\front
INFO: History tersimpan di: D:\Skripsi\Experiment\results\front_history
```

2/3 [SIDE VIEW] Melatih model (num_stages_to_freeze = 3)...

INFO:

=====

INFO: Experiment Configuration

INFO: =====

INFO: View : side

INFO: Optimizer : AdamW

INFO: Learning Rate : 5.0e-06

INFO: Weight Decay : 1.0e-02

INFO: Scheduler : ReduceLROnPlateau

INFO: Dropout : 0.6

INFO: Frozen Stages : 3

INFO: EarlyStopping : patience=5

INFO: Experiment ID : None

INFO: =====

INFO: Device: cuda

INFO: GPU: NVIDIA GeForce RTX 5060 Laptop GPU

INFO: Frame subsampling diterapkan (stride=5): 5976 frame tersisa untuk train

INFO: DataLoader dibuat: view=side split=train n=5976 batch_size=32

INFO: Frame subsampling diterapkan (stride=5): 1249 frame tersisa untuk val

INFO: DataLoader dibuat: view=side split=val n=1249 batch_size=32

INFO: Frame subsampling diterapkan (stride=5): 1215 frame tersisa untuk test

INFO: DataLoader dibuat: view=side split=test n=1215 batch_size=32

INFO: Loading pretrained weights from Hugging Face hub (timm/tf_efficientnet

INFO: HTTP Request: HEAD https://huggingface.co/timm/tf_efficientnetv2_s.in21k_ft_in1k

INFO: [timm/tf_efficientnetv2_s.in21k_ft_in1k] Safe alternative available

```

INFO: Membekukan conv_stem, bn1, dan 3 stage block pertama pada backbone
INFO: =====
INFO: STATUS GRADIENT PARAMETER MODEL (FREEZE vs TRAINABLE)
INFO: =====
INFO: backbone.conv_stem : FROZEN
INFO: backbone.bn1      : FROZEN
INFO: backbone.blocks[0] : FROZEN (contains 2 blocks)
INFO: backbone.blocks[1] : FROZEN (contains 4 blocks)
INFO: backbone.blocks[2] : FROZEN (contains 4 blocks)
INFO: backbone.blocks[3] : TRAINABLE (contains 6 blocks)
INFO: backbone.blocks[4] : TRAINABLE (contains 9 blocks)
INFO: backbone.blocks[5] : TRAINABLE (contains 15 blocks)
INFO: backbone.conv_head : TRAINABLE
INFO: backbone.bn2       : TRAINABLE
INFO: classifier         : TRAINABLE
INFO: =====
INFO: Model: tf_efficientnetv2_s | Total params: 20,180,050 | Trainable: 16,180,050
INFO: Class weights (CrossEntropyLoss): safe_driving=5.1500, phone_usage=1.0000
INFO: Label smoothing: 0.10
INFO: =====
INFO: Training: view = side | max_epochs = 30 | patience = 5
INFO: =====
INFO: Epoch 01/30 | train_loss=0.8478 | val_loss=0.8355 | val_F1=0.3000
INFO: Epoch 02/30 | train_loss=0.7981 | val_loss=0.8550 | val_F1=0.2800
INFO: Epoch 03/30 | train_loss=0.7642 | val_loss=0.8489 | val_F1=0.3000
INFO: Epoch 04/30 | train_loss=0.7238 | val_loss=0.8349 | val_F1=0.3000
INFO: Epoch 05/30 | train_loss=0.6993 | val_loss=0.7978 | val_F1=0.4000
INFO: Epoch 06/30 | train_loss=0.6562 | val_loss=0.7688 | val_F1=0.4000
INFO: Epoch 07/30 | train_loss=0.6220 | val_loss=0.7604 | val_F1=0.5000
INFO: Epoch 08/30 | train_loss=0.5977 | val_loss=0.7422 | val_F1=0.5000
INFO: Epoch 09/30 | train_loss=0.5802 | val_loss=0.7234 | val_F1=0.5000
INFO: Epoch 10/30 | train_loss=0.5555 | val_loss=0.7071 | val_F1=0.6000
INFO: Epoch 11/30 | train_loss=0.5366 | val_loss=0.6907 | val_F1=0.6000
INFO: Epoch 12/30 | train_loss=0.5211 | val_loss=0.7044 | val_F1=0.6000
INFO: Epoch 13/30 | train_loss=0.5050 | val_loss=0.6920 | val_F1=0.6000
INFO: Epoch 14/30 | train_loss=0.4860 | val_loss=0.6865 | val_F1=0.6000
INFO: Epoch 15/30 | train_loss=0.4725 | val_loss=0.6684 | val_F1=0.7000
INFO: Epoch 16/30 | train_loss=0.4685 | val_loss=0.6851 | val_F1=0.6000
INFO: Epoch 17/30 | train_loss=0.4574 | val_loss=0.6706 | val_F1=0.7000

```



```

INFO: Epoch 18/30 | train_loss = 0.4513 | val_loss = 0.6503 | val_F1 = 0.7
INFO: Epoch 19/30 | train_loss = 0.4406 | val_loss = 0.6555 | val_F1 = 0.7
INFO: Epoch 20/30 | train_loss = 0.4404 | val_loss = 0.6528 | val_F1 = 0.7
INFO: Epoch 21/30 | train_loss = 0.4289 | val_loss = 0.6347 | val_F1 = 0.7
INFO: Epoch 22/30 | train_loss = 0.4229 | val_loss = 0.6397 | val_F1 = 0.7
INFO: Epoch 23/30 | train_loss = 0.4240 | val_loss = 0.6389 | val_F1 = 0.7
INFO: Epoch 24/30 | train_loss = 0.4166 | val_loss = 0.6371 | val_F1 = 0.7
INFO: Epoch 25/30 | train_loss = 0.4113 | val_loss = 0.6339 | val_F1 = 0.7
INFO: Epoch 26/30 | train_loss = 0.4108 | val_loss = 0.6277 | val_F1 = 0.7
INFO: Epoch 27/30 | train_loss = 0.4093 | val_loss = 0.6280 | val_F1 = 0.7
INFO: Epoch 28/30 | train_loss = 0.4140 | val_loss = 0.6272 | val_F1 = 0.7
INFO: Epoch 29/30 | train_loss = 0.4043 | val_loss = 0.6251 | val_F1 = 0.7
INFO: Epoch 30/30 | train_loss = 0.4102 | val_loss = 0.6328 | val_F1 = 0.7
INFO: =====
INFO: Training selesai dalam 30.7 menit. Best epoch: 28 (val_F1 = 0.7999)
INFO: Checkpoint tersimpan di: D:\Skripsi\Experiment\checkpoints\side_history_28.pth
INFO: History tersimpan di: D:\Skripsi\Experiment\results\side_history.json

```

3/3 [FUSION & EVALUATION] Menjalankan fusion...

```

Class names: ['safe_driving', 'phone_use']
INFO: Device: cuda
INFO: Membekukan conv_stem, bn1, dan 5 stage block pertama pada backbone
INFO: =====
INFO: STATUS GRADIENT PARAMETER MODEL (FREEZE vs TRAINABLE)
INFO: =====
INFO: backbone.conv_stem : FROZEN
INFO: backbone.bn1 : FROZEN
INFO: backbone.blocks[0] : FROZEN (contains 2 blocks)
INFO: backbone.blocks[1] : FROZEN (contains 4 blocks)
INFO: backbone.blocks[2] : FROZEN (contains 4 blocks)
INFO: backbone.blocks[3] : FROZEN (contains 6 blocks)
INFO: backbone.blocks[4] : FROZEN (contains 9 blocks)
INFO: backbone.blocks[5] : TRAINABLE (contains 15 blocks)
INFO: backbone.conv_head : TRAINABLE
INFO: backbone.bn2 : TRAINABLE
INFO: classifier : TRAINABLE
INFO: =====
INFO: Model: tf_efficientnetv2_s | Total params: 20,180,050 | Trainable: 15,180,050

```

```
INFO: Loaded front checkpoint (default): epoch=29, val_F1=0.7934
INFO: Membekukan conv_stem, bn1, dan 3 stage block pertama pada backbone
INFO: =====
INFO: STATUS GRADIENT PARAMETER MODEL (FREEZE vs TRAINABLE)
INFO: =====
INFO: backbone.conv_stem : FROZEN
INFO: backbone.bn1       : FROZEN
INFO: backbone.blocks[0] : FROZEN (contains 2 blocks)
INFO: backbone.blocks[1] : FROZEN (contains 4 blocks)
INFO: backbone.blocks[2] : FROZEN (contains 4 blocks)
INFO: backbone.blocks[3] : TRAINABLE (contains 6 blocks)
INFO: backbone.blocks[4] : TRAINABLE (contains 9 blocks)
INFO: backbone.blocks[5] : TRAINABLE (contains 15 blocks)
INFO: backbone.conv_head : TRAINABLE
INFO: backbone.bn2       : TRAINABLE
INFO: classifier         : TRAINABLE
INFO: =====
INFO: Model: tf_efficientnetv2_s | Total params: 20,180,050 | Trainable: 16,180,050
INFO: Loaded side checkpoint (default): epoch=28, val_F1=0.7995
INFO: Frame subsampling diterapkan pada paired test loader (stride=5)
INFO: Paired test DataLoader: n=1215 pairs, batch_size=32
INFO: Running inference on paired test set...
INFO: Inference selesai: 1215 paired frames
INFO:
INFO: =====
INFO: TEST RESULTS: SINGLE-VIEW FRONT
INFO: =====
INFO: Accuracy : 0.8724
INFO: Precision: 0.7836 (macro)
INFO: Recall   : 0.8446 (macro)
INFO: F1-Score : 0.8074 (macro)
INFO: Confusion Matrix:
INFO:      Pred:safe Pred:phone
INFO: True:safe    177      44
INFO: True:phone   111     883
INFO: -----
INFO: =====
INFO: TEST RESULTS: SINGLE-VIEW SIDE
INFO: =====
```

INFO: Accuracy : 0.8189
INFO: Precision: 0.7116 (macro)
INFO: Recall : 0.7627 (macro)
INFO: F1-Score : 0.7301 (macro)
INFO: Confusion Matrix:
INFO: Pred:safe Pred:phone
INFO: True:safe 149 72
INFO: True:phone 148 846

INFO: -----

INFO: = = = = =

INFO: TEST RESULTS: AVERAGE FUSION (50:50)

INFO: = = = = =

INFO: Accuracy : 0.8922
INFO: Precision: 0.8116 (macro)
INFO: Recall : 0.8620 (macro)
INFO: F1-Score : 0.8328 (macro)
INFO: Confusion Matrix:
INFO: Pred:safe Pred:phone
INFO: True:safe 180 41
INFO: True:phone 90 904

INFO: -----

INFO: = = = = =

INFO: TEST RESULTS: ADAPTIVE FUSION (CONFIDENCE)

INFO: = = = = =

INFO: Accuracy : 0.8922
INFO: Precision: 0.8116 (macro)
INFO: Recall : 0.8620 (macro)
INFO: F1-Score : 0.8328 (macro)
INFO: Confusion Matrix:
INFO: Pred:safe Pred:phone
INFO: True:safe 180 41
INFO: True:phone 90 904

INFO: -----

INFO:

INFO: = = = = =

INFO: PERBANDINGAN AKHIR (Test Set — Paired Frames)

INFO: = = = = =

INFO: Method Acc Prec Rec F1

INFO: -----

INFO: Single-view FRONT	0.8724	0.7836	0.8446	0.8074
INFO: Single-view SIDE	0.8189	0.7116	0.7627	0.7301
INFO: Average Fusion (50:50)	0.8922	0.8116	0.8620	0.8322
INFO: Adaptive Fusion (confidence)	0.8922	0.8116	0.8620	0.8322
INFO: =====				
INFO: Hasil disimpan di: D:\Skripsi\Experiment\results\fusion_comparison				

✓ Re-training selesai! Seluruh hasil di folder results/ telah terupdate.

1. Load Hasil Eksperimen

In [5]:

```
# — Load semua file JSON hasil training & fusion —————
with open(RESULTS_DIR / "front_history.json") as f:
    hist_front = json.load(f)

with open(RESULTS_DIR / "side_history.json") as f:
    hist_side = json.load(f)

with open(RESULTS_DIR / "front_test_metrics.json") as f:
    metrics_front = json.load(f)

with open(RESULTS_DIR / "side_test_metrics.json") as f:
    metrics_side = json.load(f)

with open(RESULTS_DIR / "fusion_comparison.json") as f:
    fusion = json.load(f)

print(f"Front training: {len(hist_front['train_loss'])} epochs")
print(f"Side training: {len(hist_side['train_loss'])} epochs")
print(f"\nFront best epoch : {metrics_front['best_epoch']}")
print(f"Front val F1      : {metrics_front['val_macro_f1']:.4f}")
print(f"Side best epoch : {metrics_side['best_epoch']}")
print(f"Side val F1       : {metrics_side['val_macro_f1']:.4f}")
```

Front training: 30 epochs

Side training: 30 epochs

Front best epoch : 24
Front val F1 : 0.8018
Side best epoch : 23
Side val F1 : 0.8140

2. Kurva Training

In [6]:

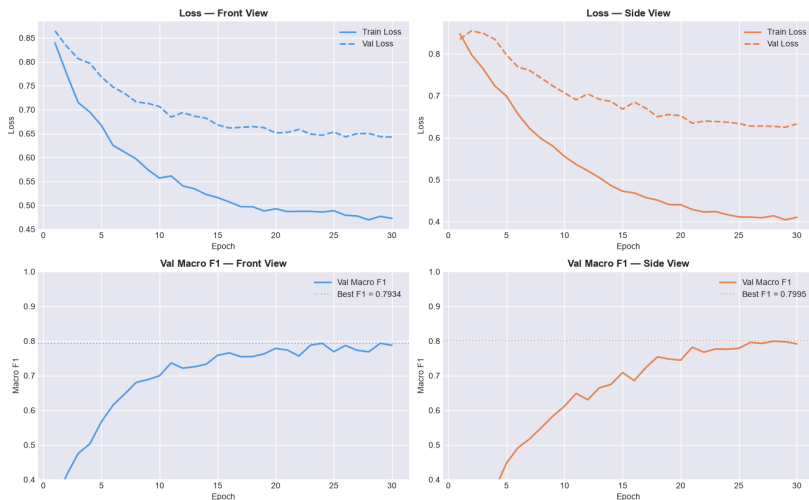
```
# — Kurva Training: Loss & F1 —————  
fig, axes = plt.subplots(2, 2, figsize=(14, 9))  
fig.suptitle("Kurva Training — Eksperimen 7", fontsize=15, fontweight="bold")  
  
for col, (hist, view) in enumerate([(hist_front, "front"), (hist_side, "side")]):  
    epochs = range(1, len(hist["train_loss"]) + 1)  
    c = COLORS[view]  
  
    # Loss  
    ax = axes[0, col]  
    ax.plot(epochs, hist["train_loss"], label="Train Loss", color=c, linewidth=2)  
    ax.plot(epochs, hist["val_loss"], label="Val Loss", color=c, linewidth=2)  
    ax.set_title(f"Loss — {view.capitalize()} View", fontweight="bold")  
    ax.set_xlabel("Epoch")  
    ax.set_ylabel("Loss")  
    ax.legend()  
  
    # F1  
    ax = axes[1, col]  
    ax.plot(epochs, hist["val_macro_f1"], label="Val Macro F1", color=c, linewidth=2)  
    ax.axhline(max(hist["val_macro_f1"]), color=c, linestyle=":", alpha=0.5,  
               label=f"Best F1 = {max(hist['val_macro_f1']):.4f}")  
    ax.set_title(f"Val Macro F1 — {view.capitalize()} View", fontweight="bold")  
    ax.set_xlabel("Epoch")  
    ax.set_ylabel("Macro F1")  
    ax.set_ylim(0.4, 1.0)  
    ax.legend()
```

```
plt.tight_layout()
```

```
plt.savefig(RESULTS_DIR / "exp7_training_curves.png", dpi=150, bbox_inches="tight")
```

```
plt.show()
print(" ✓ Disimpan ke results/exp7_training_curves.png")
```

Kurva Training — Eksperimen 7



✓ Disimpan ke results/exp7_training_curves.png

2b. Analisis Gap Train Loss vs Val Loss

Gap kecil ($\text{val_loss} - \text{train_loss} < 0.15$) mengindikasikan **overfitting berhasil diredam** oleh L2 regularization yang lebih ketat ($1e-2$) dan Label Smoothing (0.1).

In [7]:

```
# — Analisis gap train/val loss per epoch —————
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
fig.suptitle("Gap Train Loss vs Val Loss — Eksperimen 7 (Ablasi & Optimasi)")
```

```
gap_summary = []
```

```
for ax, (hist, view) in zip(axes, [(hist_front, "front"), (hist_side, "side")]):
    train_losses = np.array(hist["train_loss"])
    val_losses   = np.array(hist["val_loss"])
    gaps         = val_losses - train_losses
```

```

epochs      = range(1, len(train_losses) + 1)
best_ep_idx  = int(np.argmax(hist["val_macro_f1"]))
c = COLORS[view]

```

```

ax.plot(epochs, train_losses, label="Train Loss", color=c, linewidth=2)
ax.plot(epochs, val_losses, label="Val Loss", color=c, linewidth=2)
ax.fill_between(epochs, train_losses, val_losses, alpha=0.15, color=c)
ax.axvline(best_ep_idx + 1, color="gray", linestyle=":", linewidth=2)

```

```

final_gap = gaps[-1]
flag = "⚠ OVERFIT" if final_gap > 0.15 else ("✓ OK" if final_gap < 0.15 else "⚠")
ax.set_title(f'{view.capitalize()} View — Gap akhir: {final_gap:.4f} {flag}')
ax.set_xlabel("Epoch")
ax.set_ylabel("Loss")
ax.legend(fontsize=9)

```

```

gap_summary.append({
    "View": view.capitalize(),
    "Train Loss (akhir)": round(train_losses[-1], 5),
    "Val Loss (akhir)": round(val_losses[-1], 5),
    "Gap akhir": round(final_gap, 5),
    "Gap di best epoch": round(gaps[best_ep_idx], 5),
    "Gap maksimum": round(gaps.max(), 5),
    "Status": flag,
})

```

```

plt.tight_layout()
plt.show()

```

```

df_gap = pd.DataFrame(gap_summary).set_index("View")
display(df_gap.style.set_caption("Ringkasan Gap Loss — Eksperimen 7"))

```

```

C:\Users\adann\AppData\Local\Temp\ipykernel_10828\1890505657.py
plt.tight_layout()
d:\Skripsi\Experiment\.venv\Lib\site-packages\IPython\core\pylabtools.py
fig.canvas.print_figure(bytes_io, **kw)

```



	Train Loss (akhir)	Val Loss (akhir)	Gap akhir	Gap di best epoch	Gap maksimum	Status
--	--------------------	------------------	-----------	-------------------	--------------	--------

View

Front 0.472650 0.642760 0.170110 0.166580 0.180810

OVERFIT

Side 0.410230 0.632780 0.222550 0.213190 0.222580

OVERFIT

Ringkasan Gap Loss — Eksperimen 7

3. Confusion Matrix — Test Set

In [8]:

— Confusion Matrix: 4 metode sekaligus

CLASS_NAMES = ["safe_driving", "phone_use"]

methods = [

 ("Single-View Front", fusion["single_front"], COLORS["front"]),

 ("Single-View Side", fusion["single_side"], COLORS["side"]),

 ("Average Fusion (50:50)", fusion["average_fusion"], COLORS["avg"]),

 ("Adaptive Fusion (confidence)", fusion["adaptive_fusion"], COLORS["a"]),

]

fig, axes = plt.subplots(1, 4, figsize=(18, 4))

fig.suptitle("Confusion Matrix — Test Set (Eksperimen 7)", fontsize=14)

for ax, (title, m, color) in zip(axes, methods):

 cm = np.array(m["confusion_matrix"])

 cm_pct = cm.astype(float) / cm.sum(axis=1, keepdims=True) * 100


```

annot = np.array([[f'{v}\n({p:.1f}%)' for v, p in zip(row_v, row_p)]
                  for row_v, row_p in zip(cm, cm_pct)])

```

```

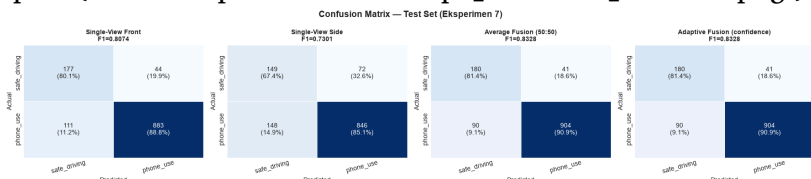
sns.heatmap(cm, annot=annot, fmt="", ax=ax, cmap="Blues",
            xticklabels=CLASS_NAMES, yticklabels=CLASS_NAMES,
            linewidths=0.5, cbar=False)
ax.set_title(f'{title}\nF1 = {m[f1_macro']:.4f}', fontweight="bold", fontstyle="italic")
ax.set_xlabel("Predicted")
ax.set_ylabel("Actual")
ax.tick_params(axis="x", rotation=15)

```

```

plt.tight_layout()
plt.savefig(RESULTS_DIR / "exp7_confusion_matrices.png", dpi=150, bbox_inches="tight")
plt.show()
print("✓ Disimpan ke results/exp7_confusion_matrices.png")

```



✓ Disimpan ke results/exp7_confusion_matrices.png

4. Tabel Perbandingan Metrik — Test Set

In [9]:

```

# — Tabel perbandingan semua metode
rows = []
for key, label in [
    ("single_front", "Single-View Front"),
    ("single_side", "Single-View Side"),
    ("average_fusion", "Average Fusion (50:50)"),
    ("adaptive_fusion", "Adaptive Fusion (confidence)"),
]:
    m = fusion[key]
    rows.append({

```

```

    "Method": label,
    "Accuracy": f'{m["accuracy"]:.4f}',
    "Precision (macro)": f'{m["precision_macro"]:.4f}',
    "Recall (macro)": f'{m["recall_macro"]:.4f}',
    "F1 (macro)": f'{m["f1_macro"]:.4f}',
})

```

```

df_summary = pd.DataFrame(rows).set_index("Method")
display(df_summary.style
    .highlight_max(axis=0, props="font-weight:bold; color:white; background-color:black")
    .set_caption("Test Set — Eksperimen 7"))

```

	Accuracy	Precision (macro)	Recall (macro)	F1 (macro)
Method				
Single-View Front	0.8724	0.7836	0.8446	0.8074
Single-View Side	0.8189	0.7116	0.7627	0.7301
Average Fusion (50:50)	0.8922	0.8116	0.8620	0.8328
Adaptive Fusion (confidence)	0.8922	0.8116	0.8620	0.8328

Test Set — Eksperimen 7

5. Bar Chart Perbandingan F1

In [10]:

```

# — Bar chart Macro F1

```

```

methods_labels = [
    "Front\n(single)",
    "Side\n(single)",
    "Average\nFusion",
    "Adaptive\nFusion",
]

```

```

]
f1_scores = [
    fusion["single_front"]["f1_macro"],

```

```

fusion["single_side"]["f1_macro"],
fusion["average_fusion"]["f1_macro"],
fusion["adaptive_fusion"]["f1_macro"],
]
bar_colors = [COLORS["front"], COLORS["side"], COLORS["avg"], COLORS["fusion"]]

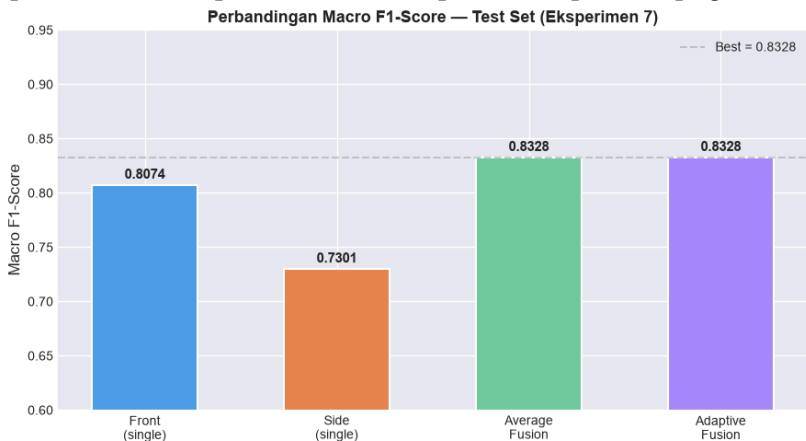
fig, ax = plt.subplots(figsize=(9, 5))
bars = ax.bar(methods_labels, f1_scores, color=bar_colors, width=0.5)

for bar, val in zip(bars, f1_scores):
    ax.text(bar.get_x() + bar.get_width() / 2, bar.get_height() + 0.003,
            f'{val:.4f}', ha="center", va="bottom", fontweight="bold", fontsize=10)

ax.set_ylim(0.6, 0.95)
ax.set_ylabel("Macro F1-Score", fontsize=12)
ax.set_title("Perbandingan Macro F1-Score — Test Set (Eksperimen 7)",
            fontsize=13, fontweight="bold")
ax.axhline(max(f1_scores), color="gray", linestyle="--", alpha=0.4, label="Best")
ax.legend(fontsize=10)

plt.tight_layout()
plt.savefig(RESULTS_DIR / "exp7_f1_comparison.png", dpi=150, bbox_inches="tight")
plt.show()
print("✓ Disimpan ke results/exp7_f1_comparison.png")

```



✓ Disimpan ke results/exp7_f1_comparison.png

6. Classification Report Per Kelas (Analisis Recall safe_driving)

In [11]:

```
# — Classification report detail per kelas
```

```
report_rows = []
```

```
for key, label in [
```

```
    ("single_front", "Front (single)"),
```

```
    ("single_side", "Side (single)"),
```

```
    ("average_fusion", "Average Fusion"),
```

```
    ("adaptive_fusion", "Adaptive Fusion"),
```

```
]:
```

```
    rep = fusion[key]["classification_report"]
```

```
    for cls in ["safe_driving", "phone_use"]:
```

```
        report_rows.append({
```

```
            "Method": label,
```

```
            "Class": cls,
```

```
            "Precision": round(rep[cls]["precision"], 4),
```

```
            "Recall": round(rep[cls]["recall"], 4),
```

```
            "F1-Score": round(rep[cls]["f1-score"], 4),
```

```
            "Support": int(rep[cls]["support"]),
```

```
        })
```

```
df_report = pd.DataFrame(report_rows)
```

```
display(df_report.style
```

```
    .background_gradient(subset=["Precision", "Recall", "F1-Score"], cmap
```

```
    .set_caption("Classification Report per Kelas — Test Set (Fokus: Recall
```

	Method	Class	Precision	Recall	F1-Score	Support
0	Front (single)	safe_driving	0.614600	0.800900	0.695500	221
1	Front (single)	phone_use	0.952500	0.888300	0.919300	994
2	Side (single)	safe_driving	0.501700	0.674200	0.575300	221
3	Side (single)	phone_use	0.921600	0.851100	0.884900	994

4 Average safe_driving 0.666700 0.814500 0.733200 221
Fusion

5 Average phone_use 0.956600 0.909500 0.932400 994
Fusion

6 Adaptive safe_driving 0.666700 0.814500 0.733200 221
Fusion

7 Adaptive phone_use 0.956600 0.909500 0.932400 994
Fusion

Classification Report per Kelas — Test Set (Fokus: Recall
safe_driving)

7. Learning Rate Schedule

In [12]:

```
# — LR Schedule kedua model —————
```

```
fig, axes = plt.subplots(1, 2, figsize=(13, 4))
```

```
fig.suptitle("Learning Rate Schedule — Eksperimen 7", fontsize=13, fontweight="bold")
```

```
for ax, (hist, view) in zip(axes, [(hist_front, "front"), (hist_side, "side")]):
```

```
    epochs = range(1, len(hist["lr"]) + 1)
```

```
    ax.plot(epochs, hist["lr"], color=COLORS[view], linewidth=2, marker="o")
```

```
    ax.set_title(f'{view.capitalize()} View', fontweight="bold")
```

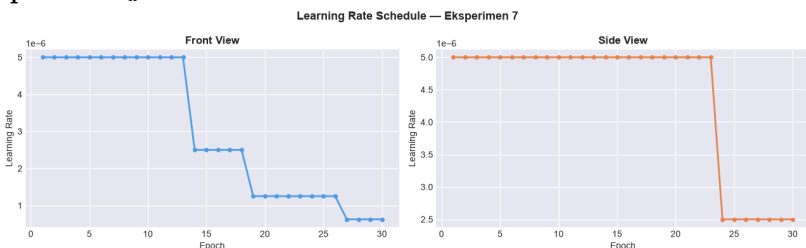
```
    ax.set_xlabel("Epoch")
```

```
    ax.set_ylabel("Learning Rate")
```

```
    ax.ticklabel_format(style="sci", axis="y", scilimits=(0, 0))
```

```
plt.tight_layout()
```

```
plt.show()
```



8. Diagnostik Kalibrasi Probabilitas

(ECE & Brier Score)

In [13]:

— *Fungsi ECE & Brier Score (reliability diagram)* —————

```
def compute_ece(probs: np.ndarray, labels: np.ndarray, n_bins: int = 10)
    """ECE = sum_b (|B_b|/n) * |accuracy(B_b) - confidence(B_b)|"""
    bin_edges = np.linspace(0.0, 1.0, n_bins + 1)
    ece = 0.0
    n = len(probs)
    for i in range(n_bins):
        lo, hi = bin_edges[i], bin_edges[i + 1]
        mask = (probs >= lo) & (probs < hi)
        if mask.sum() == 0:
            continue
        bin_acc = labels[mask].mean()
        bin_conf = probs[mask].mean()
        ece += (mask.sum() / n) * abs(bin_acc - bin_conf)
    return float(ece)
```

```
def compute_brier_score(probs: np.ndarray, labels: np.ndarray) -> float
    """BS = (1/n) * sum((p_i - y_i)^2)"""
    return float(np.mean((probs - labels) ** 2))
```

```
def reliability_diagram(ax, probs, labels, title, color, n_bins = 10):
    """Reliability diagram (calibration curve) untuk satu model."""
    bin_edges = np.linspace(0.0, 1.0, n_bins + 1)
    bin_centers, bin_acc, bin_count = [], [], []
    for i in range(n_bins):
        lo, hi = bin_edges[i], bin_edges[i + 1]
        mask = (probs >= lo) & (probs < hi)
        if mask.sum() == 0:
            continue
        bin_centers.append(probs[mask].mean())
        bin_acc.append(labels[mask].mean())
        bin_count.append(mask.sum())
```

```
bin_centers = np.array(bin_centers)
```

```
bin_acc = np.array(bin_acc)
```

```
ax.plot([0, 1], [0, 1], "k--", alpha = 0.5, label = "Perfect calibration")
ax.bar(bin_centers, bin_acc, width = 0.08, alpha = 0.5, color = color, la
ax.plot(bin_centers, bin_acc, "o-", color = color, linewidth = 2)
ax.set_xlim(0, 1)
ax.set_ylim(0, 1)
ax.set_xlabel("Mean Predicted Confidence")
ax.set_ylabel("Fraction of Positives (Accuracy)")
ax.set_title(title, fontweight = "bold")
ax.legend(fontsize = 8)
```

```
print("✓ Fungsi kalibrasi siap.")
```

✓ Fungsi kalibrasi siap.

In [14]:

```
# — Jalankan inferensi untuk mendapatkan skor probabilitas —————
import torch
from src.evaluate import load_trained_model
from src.fusion import get_paired_test_loader, get_paired_scores
```

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f'Device: {device}')
```

```
model_front, _ = load_trained_model("front", device)
model_side, _ = load_trained_model("side", device)
paired_loader = get_paired_test_loader()
```

```
scores_front, scores_side, labels = get_paired_scores(
    model_front, model_side, paired_loader, device
)
labels = labels.astype(float)
print(f'✓ Inferensi selesai: {len(labels)} sampel')
```

Class names: ['safe_driving', 'phone_use']

Device: cuda

INFO: Membekukan conv_stem, bn1, dan 5 stage block pertama pada ba

```

INFO: =====
INFO: STATUS GRADIENT PARAMETER MODEL (FREEZE vs TRAINABLE)
INFO: =====
INFO: backbone.conv_stem : FROZEN
INFO: backbone.bn1       : FROZEN
INFO: backbone.blocks[0] : FROZEN (contains 2 blocks)
INFO: backbone.blocks[1] : FROZEN (contains 4 blocks)
INFO: backbone.blocks[2] : FROZEN (contains 4 blocks)
INFO: backbone.blocks[3] : FROZEN (contains 6 blocks)
INFO: backbone.blocks[4] : FROZEN (contains 9 blocks)
INFO: backbone.blocks[5] : TRAINABLE (contains 15 blocks)
INFO: backbone.conv_head : TRAINABLE
INFO: backbone.bn2       : TRAINABLE
INFO: classifier         : TRAINABLE
INFO: =====
INFO: Model: tf_efficientnetv2_s | Total params: 20,180,050 | Trainable:
INFO: Loaded front checkpoint (default): epoch=29, val_F1=0.7934
INFO: Membekukan conv_stem, bn1, dan 3 stage block pertama pada backbone
INFO: =====
INFO: STATUS GRADIENT PARAMETER MODEL (FREEZE vs TRAINABLE)
INFO: =====
INFO: backbone.conv_stem : FROZEN
INFO: backbone.bn1       : FROZEN
INFO: backbone.blocks[0] : FROZEN (contains 2 blocks)
INFO: backbone.blocks[1] : FROZEN (contains 4 blocks)
INFO: backbone.blocks[2] : FROZEN (contains 4 blocks)
INFO: backbone.blocks[3] : TRAINABLE (contains 6 blocks)
INFO: backbone.blocks[4] : TRAINABLE (contains 9 blocks)
INFO: backbone.blocks[5] : TRAINABLE (contains 15 blocks)
INFO: backbone.conv_head : TRAINABLE
INFO: backbone.bn2       : TRAINABLE
INFO: classifier         : TRAINABLE
INFO: =====
INFO: Model: tf_efficientnetv2_s | Total params: 20,180,050 | Trainable:
INFO: Loaded side checkpoint (default): epoch=28, val_F1=0.7995
INFO: Frame subsampling diterapkan pada paired test loader (stride=5)
INFO: Paired test DataLoader: n=1215 pairs, batch_size=32

```

✓ Inferensi selesai: 1215 sampel

In [15]:

```
# — Hitung & Tampilkan ECE + Brier Score —
```

```
ece_front = compute_ece(scores_front, labels)
```

```
ece_side = compute_ece(scores_side, labels)
```

```
bs_front = compute_brier_score(scores_front, labels)
```

```
bs_side = compute_brier_score(scores_side, labels)
```

```
# Tabel ringkasan
```

```
df_calib = pd.DataFrame([
```

```
    {"View": "Front", "ECE": round(ece_front, 5), "Brier Score": round(bs_fr
```

```
    "ECE Status": "✓ BAIK" if ece_front < 0.05 else ("⚠ SEDANG" if e
```

```
    "BS Status": "✓ BAIK" if bs_front < 0.10 else ("⚠ SEDANG" if bs
```

```
    {"View": "Side", "ECE": round(ece_side, 5), "Brier Score": round(bs_si
```

```
    "ECE Status": "✓ BAIK" if ece_side < 0.05 else ("⚠ SEDANG" if e
```

```
    "BS Status": "✓ BAIK" if bs_side < 0.10 else ("⚠ SEDANG" if bs
```

```
]).set_index("View")
```

```
display(df_calib.style.set_caption("Diagnostik Kalibrasi — Test Set (Eksperimen 7)"))
```

```
# Interpretasi
```

```
print("\nInterpretasi:")
```

```
for view, ece, bs, tl in [("FRONT", ece_front, bs_front, hist_front["train_loss"]
```

```
                        ("SIDE", ece_side, bs_side, hist_side["train_loss"])[-1])
```

```
    if ece > 0.10 and tl < 0.10:
```

```
        diag = "Overconfidence kemungkinan OVERFITTING"
```

```
    elif ece < 0.05:
```

```
        diag = "Well-calibrated — kemungkinan LEGITIMATE overconfidence"
```

```
    else:
```

```
        diag = "Ambiguous — perlu ablasi regularisasi lebih lanjut"
```

```
    print(f" [{view}] train_loss = {tl:.5f}, ECE = {ece:.5f} → {diag}")
```

```
        ECE    Brier Score ECE Status  BS Status
```

```
View
```

```
Front 0.202580 0.121330    ✗ BURUK ⚠ SEDANG
```

```
Side 0.196350 0.135790    ✗ BURUK ⚠ SEDANG
```

```
Diagnostik Kalibrasi — Test Set (Eksperimen 7)
```

Interpretasi:

```
[FRONT] train_loss = 0.47265, ECE = 0.20258 → Ambiguous — perlu ablasi regularisasi lebih lanjut
```

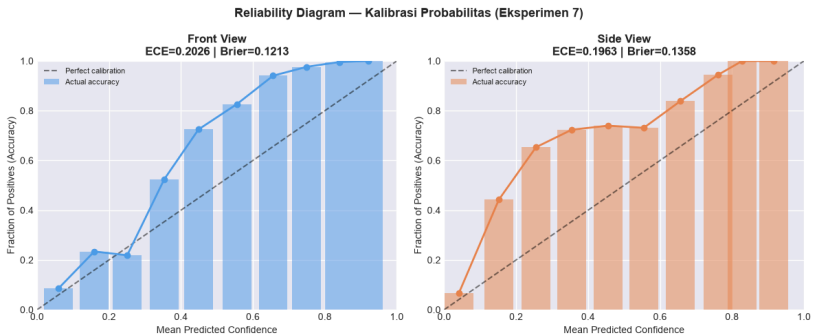
[SIDE] train_loss=0.41023, ECE=0.19635 → Ambiguous — perlu abla

In [16]:

```
# — Reliability Diagram (Calibration Curve) —————  
fig, axes = plt.subplots(1, 2, figsize=(12, 5))  
fig.suptitle("Reliability Diagram — Kalibrasi Probabilitas (Eksperimen 7)  
fontsize = 13, fontweight = "bold")
```

```
reliability_diagram(axes[0], scores_front, labels,  
f'Front View\nECE = {ece_front:.4f} | Brier = {bs_front:.4f}',  
COLORS["front"])  
reliability_diagram(axes[1], scores_side, labels,  
f'Side View\nECE = {ece_side:.4f} | Brier = {bs_side:.4f}',  
COLORS["side"])
```

```
plt.tight_layout()  
plt.savefig(RESULTS_DIR / "exp7_reliability_diagram.png", dpi = 150, bb  
plt.show()  
print("✓ Disimpan ke results/exp7_reliability_diagram.png")
```



✓ Disimpan ke results/exp7_reliability_diagram.png

9. Ringkasan Eksperimen

In [17]:

```
# — Cetak ringkasan eksperimen —————  
from src.config import WEIGHT_DECAY, LABEL_SMOOTHING, CLASS_Y  
  
print(" = " * 60)
```

```
print("RINGKASAN EKSPERIMEN 7 (Optimasi Side View)")
print("=" * 60)
```

```
print(f"\n✳ Model: EfficientNetV2-S (Binary Classification)")
print(f"  NUM_STAGES_TO_FREEZE (Front) : {NUM_STAGES_TO_FREEZE_FRONT}")
print(f"  NUM_STAGES_TO_FREEZE (Side) : {NUM_STAGES_TO_FREEZE_SIDE}")
print(f"  WEIGHT_DECAY : {WEIGHT_DECAY}")
print(f"  LABEL_SMOOTHING : {LABEL_SMOOTHING}")
print(f"  CLASS_WEIGHTS : {CLASS_WEIGHTS}")
```

```
print(f"\n📊 SINGLE-VIEW — Test Set:")
print(f"  Front : Acc={metrics_front['accuracy']:.4f} | F1={metrics_front['f1_macro']:.4f}")
print(f"  Side : Acc={metrics_side['accuracy']:.4f} | F1={metrics_side['f1_macro']:.4f}")
```

```
print(f"\n✂ FUSION — Test Set (paired frames):")
for key, label in [("average_fusion", "Average (50:50)", "adaptive_fusion"),
                   ("m = fusion[key]"):
print(f"  {label}: Acc={m['accuracy']:.4f} | F1={m['f1_macro']:.4f}")
```

```
best_key = max(fusion, key=lambda k: fusion[k]["f1_macro"])
print(f"\n🏆 Best Method: {fusion[best_key]['method']} (F1={fusion[best_key]['f1_macro']:.4f})")
print("=" * 60)
```

```
=====
RINGKASAN EKSPERIMEN 7 (Optimasi Side View)
=====
```

```
✳ Model: EfficientNetV2-S (Binary Classification)
  NUM_STAGES_TO_FREEZE (Front) : 5
  NUM_STAGES_TO_FREEZE (Side) : 3
  WEIGHT_DECAY : 0.01
  LABEL_SMOOTHING : 0.1
  CLASS_WEIGHTS : [5.15, 1.0]
```

```
📊 SINGLE-VIEW — Test Set:
  Front : Acc=0.8979 | F1=0.8375 (epoch 24)
  Side : Acc=0.8502 | F1=0.7381 (epoch 23)
```

```
✂ FUSION — Test Set (paired frames):
  Average (50:50): Acc=0.8922 | F1=0.8328
```

Adaptive : Acc = 0.8922 | F1 = 0.8328

🏆 Best Method: Average Fusion (50:50) (F1 = 0.8328)

=====

In []: